

实验 4 - 循环神经网络

数据科学与大数据技术专业

1 实验目标

- 理解循环神经网络 (RNN) 如何通过隐藏状态处理序列数据.
- 使用 NumPy 实现基础 RNN 单元、完整 RNN 前向传播、LSTM 单元和完整 LSTM 前向传播.
- 理解通过时间反向传播 (BPTT)、梯度爆炸、梯度裁剪和序列生成的基本机制.
- 完成恐龙名字字符级语言模型, 能够训练 RNN 并采样生成新名字.
- 阅读并复现 Karpathy 的 `minimalRNN.py`, 理解字符级 Vanilla RNN 的完整训练闭环.
- 掌握 one-hot 输入、softmax 交叉熵、隐藏状态续接、AdaGrad 参数更新和采样生成之间的关系.

2 实验环境

- 操作系统: Windows / Linux / macOS
- Python 3.7 及以上
- 主要依赖库: `numpy`、`matplotlib`
- 推荐开发环境: Jupyter Notebook 或 VS Code
- 实验文件:
 - `W1A1/Building_a_Recurrent_Neural_Network_Step_by_Step.ipynb`
 - `W1A2/Dinosaur_Island_Character_level_language_model.ipynb`
 - `2025-实验 4.ipynb`
 - `minimalRNN.py`

如本地环境缺少依赖, 可在终端中执行:

```
pip install numpy matplotlib
```

3 背景知识

3.1 序列建模与隐藏状态

循环神经网络用于处理文本、时间序列、语音等有顺序的数据. 与前馈网络不同, RNN 在每个时间步共享同一组参数, 并通过隐藏状态保存历史信息. 对时间步 t , 基础 RNN 的递推为:

$$a^{(t)} = \tanh(W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a) \quad (3.1)$$

$$\hat{y}^{(t)} = \text{softmax}(W_{ya}a^{(t)} + b_y) \quad (3.2)$$

其中 $x^{(t)}$ 是当前输入, $a^{(t)}$ 是隐藏状态, $\hat{y}^{(t)}$ 是当前预测.

本实验约定输入序列张量形状为:

$$x \in \mathbb{R}^{n_x \times m \times T_x} \quad (3.3)$$

其中 n_x 为输入维度, m 为 batch 大小, T_x 为序列长度.

3.2 LSTM 门控机制

基础 RNN 对长程依赖建模能力有限, 训练时容易出现梯度消失或梯度爆炸. LSTM 通过细胞状态和门控机制缓解这一问题. 单步 LSTM 的主要公式为:

$$\Gamma_f = \sigma(W_f[a^{(t-1)}; x^{(t)}] + b_f) \quad (3.4)$$

$$\Gamma_i = \sigma(W_i[a^{(t-1)}; x^{(t)}] + b_i) \quad (3.5)$$

$$\tilde{c}^{(t)} = \tanh(W_c[a^{(t-1)}; x^{(t)}] + b_c) \quad (3.6)$$

$$c^{(t)} = \Gamma_f \odot c^{(t-1)} + \Gamma_i \odot \tilde{c}^{(t)} \quad (3.7)$$

$$\Gamma_o = \sigma(W_o[a^{(t-1)}; x^{(t)}] + b_o) \quad (3.8)$$

$$a^{(t)} = \Gamma_o \odot \tanh(c^{(t)}) \quad (3.9)$$

遗忘门决定保留多少旧记忆, 输入门决定写入多少新信息, 输出门决定暴露多少细胞状态给隐藏状态.

3.3 字符级语言模型

字符级语言模型以单个字符为基本 token. 给定字符序列 $x_1, \dots, x_T$, 模型学习:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1}) \quad (3.10)$$

训练时, 输入为当前字符, 目标为下一个字符. 每个字符通常转为 one-hot 列向量, 输出层使用 softmax 得到下一个字符的概率分布.

对一段序列, 交叉熵损失为:

$$\mathcal{L} = - \sum_{t=1}^T \log \hat{p}_{y^{(t)}}^{(t)} \quad (3.11)$$

推理时, 模型从起始输入开始, 每一步根据 softmax 概率采样一个字符, 再把该字符作为下一步输入, 循环生成文本.

3.4 BPTT 与梯度裁剪

RNN 的反向传播需要沿时间轴从后向前展开, 简称 BPTT. 因为隐藏状态递归依赖过去, 当前时间步的损失会影响前面许多时间步的参数梯度.

长序列训练时梯度可能指数级增大. 梯度裁剪将梯度限制在固定范围内:

$$g \leftarrow \text{clip}(g, -\tau, \tau) \quad (3.12)$$

本实验中恐龙名字模型和 minimalRNN 补充实验都使用裁剪阈值 5.

3.5 AdaGrad 参数更新

minimalRNN.py 使用 AdaGrad 更新参数. 对参数 θ 和梯度 g :

$$G \leftarrow G + g^2 \quad (3.13)$$

$$\theta \leftarrow \theta - \eta \frac{g}{\sqrt{G + \epsilon}} \quad (3.14)$$

其中 G 是历史平方梯度累积缓存. AdaGrad 会自动缩小频繁更新参数的有效学习率, 有助于稳定小型 RNN 训练.

4 实验步骤

实验 4.1: RNN 与 LSTM 前向传播 (W1A1)

实验目标: 使用 NumPy 手写基础 RNN 和 LSTM 的前向传播, 明确每个时间步的输入、隐藏状态、输出和 cache.

4.1.1 RNN 单元

练习 1: rnn_cell_forward

实现单步 RNN 前向传播:

$$a_{\text{next}} = \tanh(W_{aa}a_{\text{prev}} + W_{ax}x_t + b_a) \quad (4.1)$$

$$y_t = \text{softmax}(W_{ya}a_{\text{next}} + b_y) \quad (4.2)$$

函数返回 a_{next} 、 y_t 和 $\text{cache}=(a_{\text{next}}, a_{\text{prev}}, x_t, \text{parameters})$.

4.1.2 完整 RNN 前向传播

练习 2: rnn_forward

对 T_x 个时间步循环调用 rnn_cell_forward. 实现要点:

- 从 $x.\text{shape}$ 获取 n_x, m, T_x .

- 根据 `parameters['Wya']` 获取 n_y, n_a .
- 初始化 `a` 和 `y_pred` 为零.
- 用 `a0` 初始化 `a_next`.
- 每步保存隐藏状态、预测结果和 `cache`.

4.1.3 LSTM 单元

练习 3: `lstm_cell_forward`

将 a_{prev} 与 x_t 沿行方向拼接后, 依次计算遗忘门、输入门、候选细胞状态、输出门, 最终得到 `c_next` 和 `a_next`. 函数返回:

$$a_{\text{next}}, c_{\text{next}}, y_{\text{t_pred}}, \text{cache} \quad (4.3)$$

注意 `cache` 中应保留所有门控变量, 以便选做反向传播使用.

4.1.4 完整 LSTM 前向传播

练习 4: `lstm_forward`

对 T_x 个时间步循环调用 `lstm_cell_forward`. 需要同时维护隐藏状态 `a_next` 与细胞状态 `c_next`. 返回顺序为:

$$a, y, c, \text{caches} \quad (4.4)$$

这与 `rnn_forward` 的返回值不同, 请严格按 Notebook 要求实现.

4.1.5 反向传播 (选做)

Notebook 中提供了 RNN 和 LSTM 反向传播框架. 选做内容包括:

- `rnn_cell_backward` 与 `rnn_backward`.
- `lstm_cell_backward` 与 `lstm_backward`.

完成时要注意时间维度上的梯度累加, 尤其是 `da_prev` 和 `dc_prev` 如何从未来时间步传回当前时间步.

实验 4.2: 恐龙名字字符级语言模型 (W1A2)

实验目标: 训练 RNN 生成具有恐龙名字风格的新字符串.

4.2.1 数据预处理

读取 `dinos.txt`, 统计字符表, 建立 `char_to_ix` 和 `ix_to_char`. 每个名字作为一个训练样本, 输入序列首位补 `None`, 标签序列在末尾追加换行符 `\n`.

4.2.2 梯度裁剪

练习 1: clip

对梯度字典中的 `dWaa`、`dWax`、`dWya`、`db`、`dby` 做原地裁剪:

$$g_{ij} \leftarrow \min(\max(g_{ij}, -\text{maxValue}), \text{maxValue}) \quad (4.5)$$

推荐使用 `np.clip(gradient, -maxValue, maxValue, out=gradient)`.

4.2.3 字符采样

练习 2: sample

采样流程:

1. 初始化输入 x 和隐藏状态 a_{prev} 为零.
2. 单步前向传播得到 softmax 概率.
3. 使用 `np.random.choice` 按概率采样字符 `id`.
4. 将采样字符转为 one-hot 作为下一步输入.
5. 采样到换行符或达到最大长度时停止.

4.2.4 优化步骤与训练

练习 3: optimize

整合一步训练:

$$\text{forward} \rightarrow \text{backward} \rightarrow \text{clip} \rightarrow \text{update} \quad (4.6)$$

函数返回当前损失、裁剪后的梯度和最后隐藏状态.

练习 4: model

完成训练循环:

- 循环选择一个恐龙名字.
- 构造 `X=[None]+name_indices` 和 `Y=name_indices+[newline]`.
- 调用 `optimize` 更新参数.
- 使用指数平滑损失观察训练趋势.
- 每隔一定迭代次数采样生成名字.

实验 4.3: 从零实现 Karpathy minimalRNN(2025 补充实验)

实验目标: 把 `minimalRNN.py` 拆成可测试的课堂练习, 完整实现字符级 Vanilla RNN 的训练闭环.

4.3.1 字符词表与参数初始化

- 练习 1: 使用 `sorted(set(data))` 构建字符表、`char_to_ix`、`ix_to_char`.
- 练习 2: 初始化 `Wxh`、`Whh`、`Why`、`bh`、`by`. 权重为小随机数, 偏置为零.

4.3.2 前向传播与 BPTT

- 练习 3: 实现 `one_hot` 和数值稳定的 `softmax`.
- 练习 4: 实现 `rnn_forward`, 保存 `xs`、`hs`、`ys`、`ps`, 并累加负对数似然损失.
- 练习 5: 实现 `rnn_backward`. 关键是从最后一个时间步反向遍历, 用 `dhnnext` 把未来梯度传回当前时间步.

4.3.3 训练稳定性与采样

- 练习 6: 实现 `clip_gradients` 和 `lossFun`.
- 练习 7: 实现 `sample`, 从模型概率分布中逐字符采样.
- 练习 8: 实现 `train_rnn`, 包括数据指针移动、隐藏状态续接、平滑损失、AdaGrad 更新和定期采样.

完成后应比较训练前随机采样与训练后采样的差异, 并说明模型是否学到了换行、元音、常见名字片段等字符规律.

5 核心函数对照总结

函数	所在实验	作用
<code>rnn_cell_forward</code>	W1A1	单步 RNN 前向传播
<code>rnn_forward</code>	W1A1	完整 RNN 前向传播
<code>lstm_cell_forward</code>	W1A1	单步 LSTM 前向传播
<code>lstm_forward</code>	W1A1	完整 LSTM 前向传播
<code>clip</code>	W1A2	裁剪 RNN 梯度
<code>sample</code>	W1A2 / 2025	字符级采样生成
<code>optimize</code>	W1A2	一步训练更新
<code>model</code>	W1A2	恐龙名字模型训练循环
<code>lossFun</code>	2025	minimalRNN 核心损失与梯度函数
<code>train_rnn</code>	2025	有限步 minimalRNN 训练闭环

6 实验作业

1. 完成 W1A1 中 RNN 与 LSTM 前向传播相关函数, 并通过测试单元格.
2. 完成 W1A2 中 `clip`、`sample`、`optimize`、`model`, 训练恐龙名字生成模型.
3. 完成 2025-实验 4.ipynb, 说明 `minimalRNN.py` 中前向传播、BPTT、梯度裁剪、AdaGrad 更新和采样生成分别对应 Notebook 中哪些函数.

4. 记录至少 5 个模型生成的名字样本, 分析其合理性和不足.
5. **选做:** 完成 W1A1 中 RNN/LSTM 反向传播函数.
6. **选做:** 为 `sample` 加入 `temperature` 参数, 对比不同温度下生成文本的多样性.
7. **选做:** 将 2025-实验 4.ipynb 中的小模型迁移到更大的纯文本语料上, 分析训练损失和生成样例的变化.

7 提交要求

请在提交前检查材料是否齐全. 本课程作业上传只接受文本文件, 不接受 PDF、Word 文档、图片、截图、压缩包等二进制文件. 采用这一规则, 一方面是为了引入 AI 辅助评阅, 另一方面也是希望大家养成用清晰文本表达实验过程和结论的习惯.

如果实验报告确实需要配图, 请优先思考是否可以用文字、公式或表格表达清楚; 若必须插图, 请将图片放入 Notebook 中, 形成一份完整的 .ipynb 实验报告. 若使用 $\text{\LaTeX}$ 或 Word 撰写报告, 请使用 `pandoc` 等工具转换为 .md 或 .txt 等文本格式后再提交, 不要提交 .pdf 或 .docx.

1. Jupyter Notebook 文件 (.ipynb)

- (a) 必交内容包括本实验要求完成的 Notebook 文件, 至少包含 W1A1、W1A2 和 2025-实验 4.ipynb.
- (b) Notebook 中应包含完整代码、测试结果、训练输出、生成样例和必要分析文字.
- (c) 只需提交 Notebook 源文件, 不需要额外导出为 PDF、HTML、Python 脚本、Word 文档、截图或图片文件.

2. 实验过程与 AI 互动记录文本文件 (.md / .txt)

- (a) 必须额外提交一份文本文件, 作为实验过程与 AI 互动记录, 推荐使用 .md 或 .txt.
- (b) 记录内容至少包括: 查阅了哪些资料、向 AI 提出了哪些问题、如何继续思考、调试、修改代码或完善实验分析.
- (c) 不要把 AI 的回复原文放入记录中, 也不要复制粘贴大段代码; 记录应只包含你自己写的内容, 包括你的问题、判断、尝试、修改思路 and 必要的简短总结.
- (d) 记录应尽量按时间顺序整理, 不要只保留最终答案.

8 关于作业的重要说明

本实验的核心目标是理解序列模型的递推结构和训练闭环, 而不是只得到一份能运行的代码. 代码填空部分应由学生自己完成, 并通过调试和单元测试逐步确认正确性.

可以使用 AI 工具辅助理解公式、解释报错、查询 NumPy API 或检查矩阵维度, 但不得直接让 AI 代写全部必做代码后不加理解地提交. 若使用 AI 工具, 必须在文本记录中如实写明自己的提问、自己的判断和后续修改过程.

AI 互动记录应只保留学生自己写的内容. 不要粘贴 AI 的完整回复, 不要粘贴复制来的大段代码, 也不要把聊天记录原样导出后提交.

选做内容可以更自由地使用 AI 工具, 不需记录使用过程.